Data Wrangling for Visualization (R)

Workshop 4 — Module 2

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

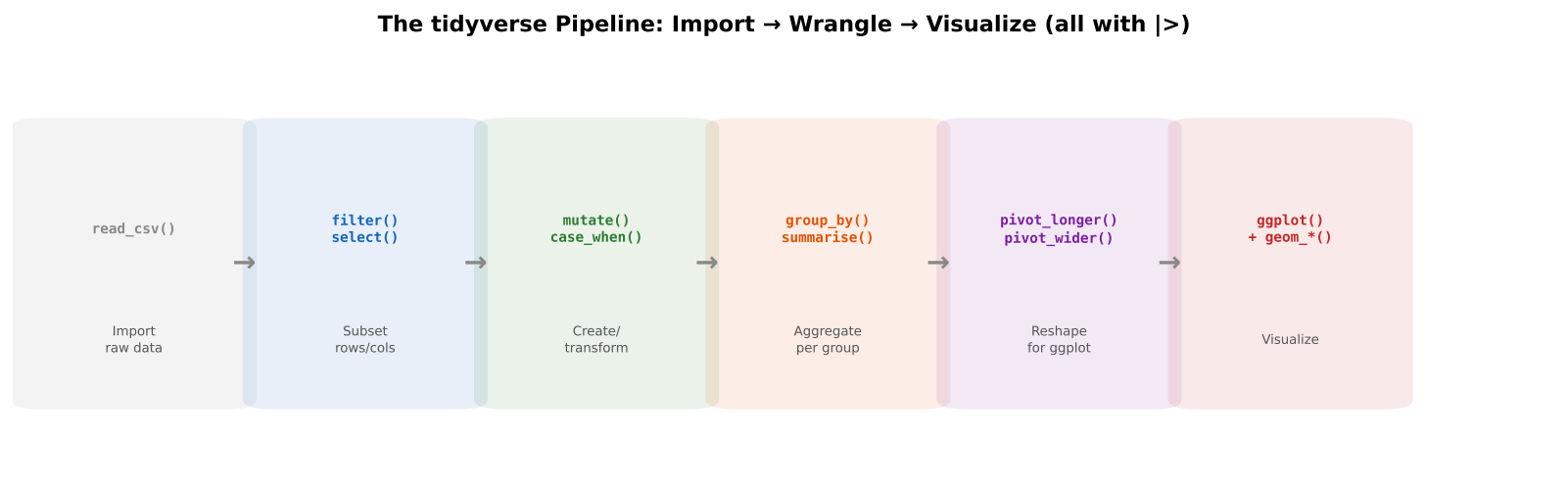
Module Roadmap

1. The tidyverse pipeline: import $\rightarrow$ wrangle $\rightarrow$ visualize
2. The pipe operator: |> reads top-to-bottom
3. Six dplyr verbs: filter, select, mutate, arrange, group_by, summarise
4. Reshaping with tidyr: pivot_longer, pivot_wider
5. Common cleaning patterns for visualization
6. Pipe-to-plot: wrangling directly into ggplot

Learning Outcome
You will wrangle any messy CSV into tidy, ggplot-ready form using a single piped chain of dplyr and tidyr operations.

The tidyverse Pipeline

Import → Wrangle → Visualize



The Pipe Operator

The Pipe Operator: Read Top-to-Bottom, Not Inside-Out

WITHOUT pipe (nested)

```
arrange(
summarise(
  group_by(
filter(apps, Rating > 4),
  Category),
mean_r = mean(Rating)),
desc(mean_r))
```

WITH pipe (|> or %>%)

```
apps |>
filter(Rating > 4) |>
group_by(Category) |>
summarise(mean_r = mean(Rating)) |>
arrange(desc(mean_r))
```

Pro-Tip

R 4.1+ introduced the native pipe |>. The tidyverse pipe %>% (from magrittr) still works identically. Both pass the left-hand result as the first argument of the right-hand function. Read the chain top-to-bottom: "take apps, then filter, then group, then summarise, then arrange."

Six dplyr Verbs

The Grammar of Data Wrangling

Six dplyr Verbs: The Grammar of Data Wrangling

Verb	What It Does	Example Code	Effect
`filter()`	Keep rows that match a logical condition	`apps |> filter(Rating > 4, Type == 'Free')`	**Rows: subset**
`select()`	Keep, drop, or rename columns	`apps |> select(App, Rating, Reviews, Type)`	**Cols: subset**
`mutate()`	Create new columns or transform existing ones	`apps |> mutate( log_reviews = log10(Reviews))`	**Cols: create**
`arrange()`	Sort rows by one or more columns	`apps |> arrange( desc(Rating))`	**Rows: sort**
`group_by()`	Set grouping variable for subsequent operations	`apps |> group_by(Category)`	**Structure**
`summarise()`	Compute one summary per group	`apps |> summarise( mean_r = mean(Rating))`	**Rows: collapse**

filter() + select(): Subset Rows and Columns

```r
# filter(): keep rows matching condition
apps |>
  filter(Rating > 4.0,                # AND: comma = &
         Type == "Free",
         Category %in% c("GAME", "FAMILY", "TOOLS"))

# select(): keep, drop, or rename columns
apps |>
  select(App, Category, Rating, Reviews, Type) |>  # keep 5 cols
  select(-Reviews) |>                               # drop 1 col
  rename(app_name = App)                            # rename

# Helper functions inside select():
apps |> select(starts_with("Content"),   # prefix match
               ends_with("Rating"),       # suffix match
               where(is.numeric))         # type-based
```

mutate() + case_when(): Create and Transform

```r
# mutate(): add or transform columns
apps |>
  mutate(
    Reviews = as.numeric(Reviews),                # type conversion
    log_reviews = log10(Reviews + 1),             # log transform
    Size_MB = str_remove(Size, "M") |>            # string cleaning
      as.numeric(),
    date = mdy(`Last Updated`),                   # date parsing
    year = year(date),                            # extract component
    price_cat = case_when(                        # conditional recode
      Price == 0 ~ "Free",
      Price < 5  ~ "Under $5",
      Price < 20 ~ "$5-$20",
      TRUE       ~ "$20+"
    )
  )

# across(): apply a function to multiple columns at once
apps |> mutate(across(where(is.character), str_trim))
```

group_by() + summarise(): Aggregate per Group

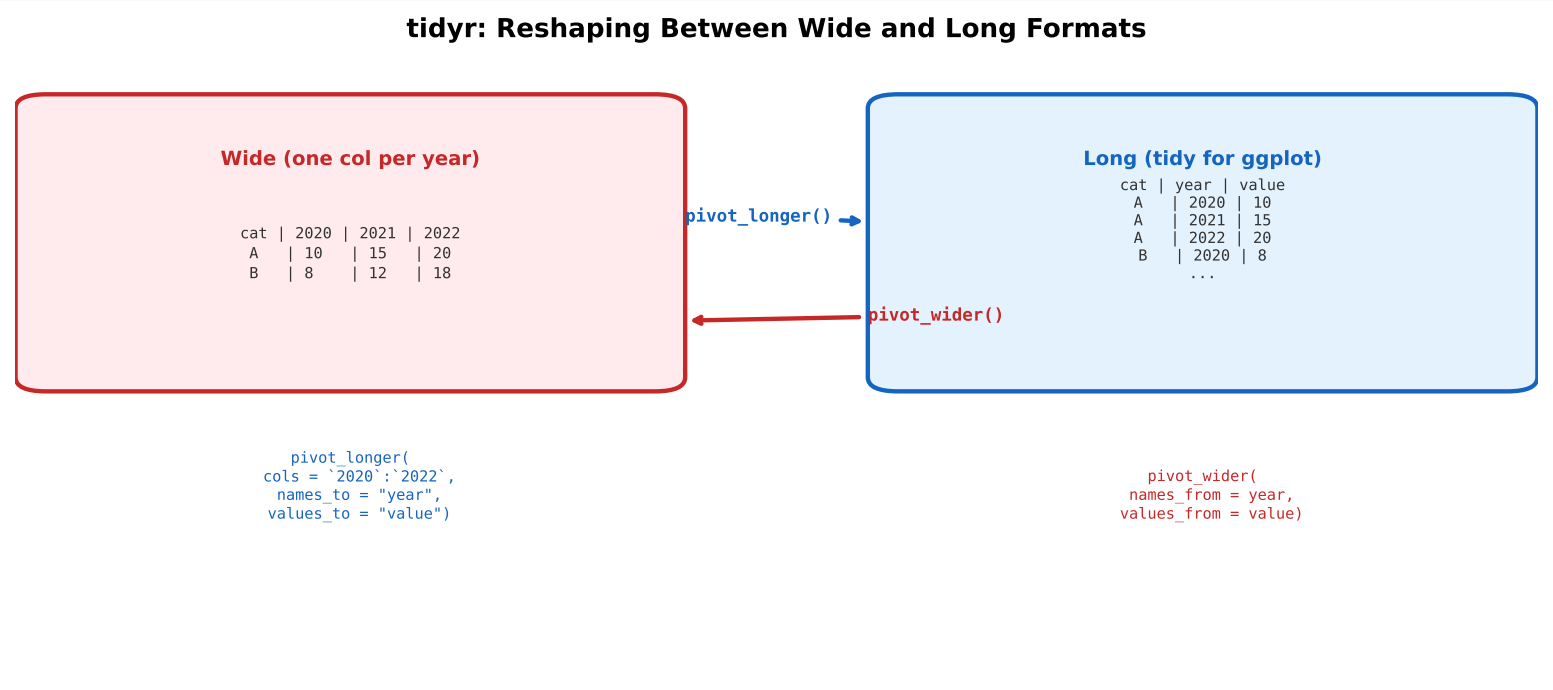
```r
# group_by() + summarise(): one row per group
apps |>
  filter(!is.na(Rating)) |>
  group_by(Category, Type) |>            # two-variable grouping
  summarise(
    n        = n(),                       # count
    mean_r   = mean(Rating),              # mean
    median_r = median(Rating),            # median
    sd_r     = sd(Rating),                # standard deviation
    q25      = quantile(Rating, 0.25),    # 25th percentile
    .groups  = "drop"                     # ungroup after
  ) |>
  arrange(desc(mean_r))                   # sort by mean (descending)

# Quick counting shortcut
apps |> count(Category, sort = TRUE)      # = group_by + summarise(n=
    n())

# slice_max/min: keep top/bottom n rows
apps |> slice_max(Rating, n = 10)         # top 10 by Rating
```

Reshaping with tidyr

pivot_longer ↔ pivot_wider



Data Insight

ggplot2 requires **long (tidy)** format: each variable is a column, each observation is a row. If your data has years as columns (wide), you must `pivot_longer()` before plotting. The reverse (`pivot_wider()`) is for creating summary tables.

Reshaping in Practice

```r
# WIDE -> LONG (for ggplot)
df_wide |>
  pivot_longer(
    cols = `2020`:`2024`,          # columns to stack
    names_to = "year",              # new column for former col names
    values_to = "revenue"           # new column for values
  ) |>
  mutate(year = as.integer(year)) |>
  ggplot(aes(x = year, y = revenue, color = region)) +
  geom_line()

# LONG -> WIDE (for summary tables)
df_long |>
  pivot_wider(
    names_from = year,
    values_from = revenue
  )

# separate(): split one column into two
apps |> separate(Genres, into = c("genre1", "genre2"), sep = ";")

# separate_rows(): one row per value (explode)
netflix |> separate_rows(listed_in, sep = ", ")
```

Cleaning Patterns

Six Common Cleaning Patterns

Six Common Data Cleaning Patterns in R

Pattern	R Code	Purpose
Missing values	`drop_na(Rating)` `replace_na(list(col = 0))`	Remove or impute
Type conversion	`mutate(Reviews =` `  as.numeric(Reviews))`	Character → numeric
String cleaning	`mutate(Size = str_remove(` `  Size, 'M') |> as.numeric())`	Remove units, parse
Date parsing	`mutate(date = mdy(` ` `` `Last Updated` ``))`	lubridate::mdy()

Recoding	`mutate(type2 = case_when(`
	`  Price == 0 ~ 'Free',`
	`  TRUE ~ 'Paid'))`
		Create derived factors

Factor ordering	`mutate(Category =` `  fct_reorder(Category, n))`	Sort for ggplot

Cleaning the Google Play Store Dataset

```r
apps_clean <- read_csv("googleplaystore.csv") |>
  # 1. Remove duplicates
  distinct(App, .keep_all = TRUE) |>
  # 2. Fix types
  mutate(
    Reviews  = as.numeric(Reviews),
    Installs = str_remove_all(Installs, "[+,]") |> as.numeric(),
    Size     = case_when(
      str_detect(Size, "M") ~ str_remove(Size, "M") |> as.numeric(),
      str_detect(Size, "k") ~ str_remove(Size, "k") |> as.numeric()
          / 1024,
      TRUE ~ NA_real_),
    Price    = str_remove(Price, "\\$") |> as.numeric(),
    date     = mdy(`Last Updated`)
  ) |>
  # 3. Filter valid
  filter(!is.na(Rating), Type %in% c("Free", "Paid")) |>
  # 4. Factor ordering
  mutate(Category = fct_lump_n(Category, 8) |> fct_infreq())
```

Joins

Combining Tables with Joins

Four Join Types in dplyr

Function	Keeps	Use Case
`left_join(x, y)`	All x + matching y	Most common for viz
`inner_join(x, y)`	Only matching rows	Strictest: no NAs
`full_join(x, y)`	All rows from both	Exploratory: see everything
`anti_join(x, y)`	x rows with NO match in y	Find missing data

Pro-Tip

`left_join()` is the default for visualization: keep all rows from your main table, add columns from a lookup table. `anti_join()` is invaluable for finding "what2019s missing" — rows in table A with no match in table B.

Pipe-to-Plot

One Chain: Raw Data → Chart

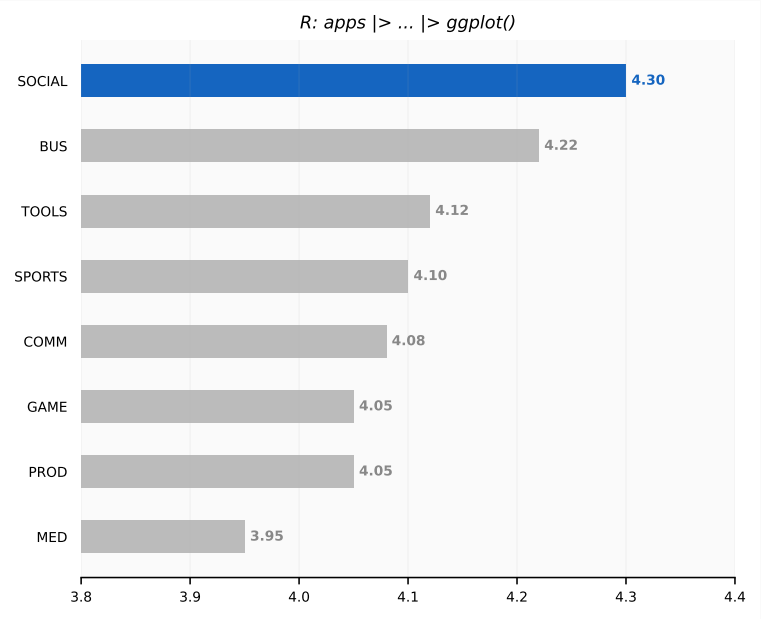
Pipe-to-Plot: One Continuous Chain from Raw Data to Chart

```
apps |>
filter(!is.na(Rating), Type %in% c("Free","Paid")) |>
mutate(Reviews = as.numeric(Reviews),
log_reviews = log10(Reviews + 1)) |>
group_by(Category) |>
summarise(mean_rating = mean(Rating),
n = n(), .groups = "drop") |>
slice_max(n, n = 8) |>
mutate(Category = fct_reorder(Category, mean_rating)) |>
ggplot(aes(x = Category, y = mean_rating)) +
geom_col(fill = "#1565C0", width = 0.5) +
coord_flip() +
theme_minimal() +
labs(title = "Mean Rating by Top 8 Categories")
```

Complete Examples

R: Full Wrangling → ggplot Chain

```
apps_clean |>
  group_by(Category) |>
  summarise(
    mean_r = mean(Rating),
    n = n(),
    .groups = "drop") |>
  slice_max(n, n = 8) |>
  mutate(Category =
    fct_reorder(Category, mean_r),
    hl = mean_r == max(mean_r)) |>
  ggplot(aes(x = Category,
    y = mean_r, fill = hl)) +
  geom_col(width = 0.5,
    show.legend = FALSE) +
  geom_text(aes(
    label = round(mean_r, 2)),
    hjust = -0.2, size = 3,
    fontface = "bold") +
  scale_fill_manual(values =
    c("TRUE" = "#1565C0",
      "FALSE" = "#BBBBBB")) +
  coord_flip() +
  theme_minimal() +
  labs(title = "Mean Rating
    by Top Categories",
    x = NULL, y = "Mean Rating")
```

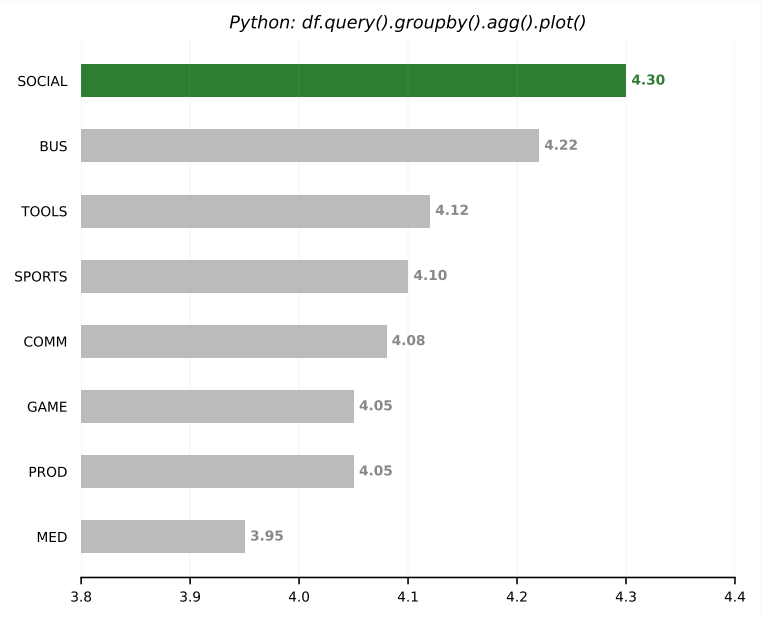


The entire pipeline — from grouped summary to finished chart — is a single piped chain. No intermediate variables needed.

Python Equivalent: pandas Chain

```python
(apps_clean
    .groupby("Category")
    .agg(
        mean_r=("Rating", "mean"),
        n=("Rating", "count"))
    .nlargest(8, "n")
    .sort_values("mean_r")
    .assign(color=lambda d:
        np.where(d["mean_r"] ==
            d["mean_r"].max(),
            "#1565C0", "#BBBBBB"))
    .plot.barh(
        y="mean_r",
        color="color",  #
            simplified
        figsize=(6, 5),
        legend=False)
)

# Or manual for full control:
summary = (apps_clean
    .groupby("Category")
    .agg(mean_r=("Rating","mean"),
         n=("Rating","count"))
    .nlargest(8, "n")
    .sort_values("mean_r"))
```



pandas uses method chaining (`.groupby().agg().nlargest()`) as the equivalent of RŽ019s pipe. The `.assign()` method creates columns inline.

Synthesis

Key Takeaways

1. The **pipe** (|>) transforms nested code into readable top-to-bottom chains.
2. **Six dplyr verbs** handle 90% of wrangling: filter, select, mutate, arrange, group_by, summarise.
3. **pivot_longer()** is required before plotting wide data in ggplot2; **pivot_wider()** creates summary tables.
4. Common cleaning: `drop_na()`, `as.numeric()`, `str_remove()`, `mdy()`, `case_when()`, `fct_reorder()`.
5. **Pipe-to-plot**: wrangle and plot in a single chain with no intermediate variables.
6. `left_join()` adds context; `anti_join()` finds missing data.

Essential Readings

- Wickham, H. & Grolemund, G. (2023). *R for Data Science*, 2nd ed., Chapters 3–5 (Data Transformation, Tidy Data).
- Wickham, H. (2014). "Tidy Data." *Journal of Statistical Software*, 59(10).
- tidyverse documentation: `https://www.tidyverse.org`

Next Module: Data Wrangling for Visualization in Python (W04-M03)